



Indian Institute of Technology Bhilai

Department of Mathematics

Graph Sparsification for High-Performance Graph Neural Networks

Project Supervisor

Dr. Kuldeep Kumar Kataria

Project Co-Supervisor

Dr. Vishwesh Jatala

Submitted by:

Himanshu (S24MA006)

Program/Dept.: M.Sc. Mathematics and Computing

Himanshu

Abstract

Graph Neural Networks (GNNs) have gained considerable attention as an effective tool for learning over graph-structured data. As the scale of real-world graphs continues to grow, the computational cost of GNN training has become a critical concern, particularly for graphs containing millions of nodes and billions of edges. Existing sparsification approaches, such as random sampling and graph coarsening, attempt to mitigate this cost but fall short in one important aspect — they fail to account for the spectral properties of the graph and the structural role of individual nodes.

In this work, we propose a centrality-driven edge sparsification method that preserves the most structurally significant connections while discarding redundant ones. Node importance is measured using Eigenvector Centrality, and edges connecting less important nodes are pruned subject to a connectivity-preservation constraint. The resulting sparser graph retains the essential message-passing behavior of the original while significantly reducing computational overhead.

We implemented and evaluated the proposed method on several large-scale benchmark datasets. Experimental results show that GNN models trained on sparsified graphs achieve training speedups without meaningful degradation in classification accuracy, demonstrating a strong trade-off between efficiency and performance.

Contents

Abstract	1
1 Introduction	1
2 Background	3
2.1 Adjacency Matrix Representation	7
2.2 Adjacency List Representation	8
2.3 Coordinate List (COO) Representation	8
2.4 Compressed Sparse Row (CSR)	9
2.5 Centrality Measures	11
2.6 Message Passing	13
2.7 Illustrative Example of Message Passing in GCN	13
2.8 GraphSAGE	16
2.9 Graph Attention Network	16
3 Approach	17
3.1 Scalability of Eigenvector Centrality for Graph Sparsification	17
3.2 Node Importance via Eigenvector Centrality	18
3.3 Algorithm Outline	20
3.4 Advantages of the Approach	21
3.5 Time Complexity Analysis: Original Graph vs Sparsified Graph	21
3.6 Message Passing Error after Edge Sparsification	23
4 Datasets and Experimental Setup	26
4.1 Dataset Statistics	26
4.2 Experimental Setup	27
4.3 Average Degree Sparsification	28
4.4 Results: Top-K Sparsification	28
4.5 Results: Threshold-based Sparsification	29
4.6 Effect of Edge Sparsification on Model Accuracy	30
4.7 Code Availability	32
5 Related Work	33
5.1 Classical Graph Sparsification	33

5.2	Sampling-Based Methods for Scalable GNN Training	34
5.3	Centrality-Based Graph Reduction	35
5.4	Summary and Positioning of Our Work	36
6	Conclusion	37
6.1	Methodological Contributions	37
6.2	Computational Efficiency	37
6.3	Empirical Observations	38
	Reference	41

Chapter 1

Introduction

Most of the real-world data can be represented using the graph data structures. As a result, for many decades, graph analytics [13] has become an active area of research to solve many problems dealing with graph data. Recently, Graph Neural Networks (GNN) [7] have emerged as a machine learning toolbox to solve complex problems on the graph data structure. Subsequently, GNNs have been adopted in various applications [19], such as social networks [18], recommendation systems [20], knowledge graph [10], financial fraud detection [8] and medicine [11] etc.

Graph-structured data has become central to a wide range of real-world problems, spanning social platforms, citation databases, e-commerce systems, and biological research. In all these domains, the relationships between entities are as meaningful as the entities themselves. Graph Neural Networks (GNNs) were built to exploit precisely this relational structure. Unlike conventional neural networks that assume independence between samples, GNNs operate directly on graphs and allow each node to aggregate information from its neighborhood — a process called message passing. This mechanism enables GNNs to learn representations sensitive to both local and global graph structure, yielding strong performance on tasks such as node classification, link prediction, and graph-level inference.

However, scaling GNNs to real-world graphs poses serious computational challenges. Since message passing is inherently edge-dependent, the cost of a single GNN layer is $O(|E|.d)$, where $|E|$ is the number of edges and d is the feature dimension. For large graphs — such as Reddit with 57 million edges or OGBN-Products with 63 million — this becomes a major bottleneck even on high-end GPU hardware. Graph sparsification offers a natural remedy: by deliberately removing less important edges, one can produce a leaner graph on which GNN training is faster and cheaper, while preserving the graph’s essential structural properties. The key challenge is determining which edges to remove, since naive or random pruning risks discarding structurally critical connections and degrading model performance.

This work takes approach. Rather than treating edges as isolated objects to be sampled or pruned based on local statistics alone, we anchor our sparsification strategy in a global measure of node importance derived from spectral graph theory — specifically, eigenvector centrality. The central intuition is straightforward: an edge is important if it connects nodes that are

themselves important, and a node is important if it is connected to many other important nodes. This recursive definition is precisely what eigenvector centrality captures. By computing centrality scores for all nodes and using them to rank edges, we obtain a principled, theoretically motivated criterion for deciding which edges to keep and which to remove. The resulting framework is both computationally tractable for large graphs and grounded in well-established mathematical theory, offering a stronger foundation than heuristic alternatives.

To summarize, this project makes the following contributions:

- We propose a novel graph sparsification approach to improve the performance and scalability of Graph Neural Network (GNN) training. The key idea is to reduce the size of the graph while preserving its structural and spectral properties along with important node features.
- We implement the proposed sparsification techniques in Python and evaluate their effectiveness on large-scale graph datasets.
- We demonstrate that the proposed sparsification methods achieve significant speedup in GNN training while maintaining comparable predictive performance relative to training on the original full graph.

We organize the rest of the report as follows.

Chapter 1 introduces the fundamentals of graph data structures and Graph Neural Networks. Chapter 2 discusses background concepts including graph representations, centrality measures, and message passing mechanisms. Chapter 3 presents our proposed graph sparsification methodology and algorithmic framework. Chapter 4 describes the datasets, experimental setup, and performance evaluation of the proposed methods. Chapter 5 reviews existing literature related to graph sparsification and GNN acceleration. Finally, Chapter 6 concludes the report with key findings and future directions.

Chapter 2

Background

Definition 2.0.1 (Graph). : A graph is a pair of two sets V and E where V is the collection of vertices (nodes) and E is the collection of edges connecting the nodes. Generally the graph is denoted by

$$G = (V, E).$$

A vertex (or node) is a crucial unit of a graph representing an entity in the network. The collection of vertices is denoted by V .

In a graph, an edge is a link between two vertices. If an edge connecting vertex u and vertex v . Then it can be written as

$$(u, v) \in E$$

where E is the collection of edges.

Example 2.0.2. If two users in a social network are friends, there is an edge between them.

Example 2.0.3. In a citation network, each research paper can be represented as a vertex.

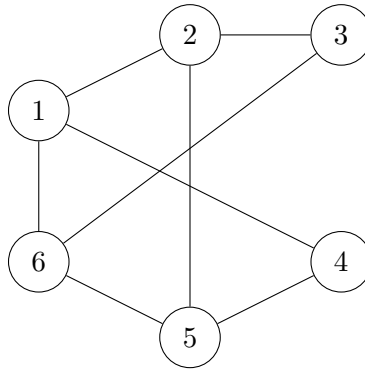


Figure 2.1: Example Graph with 6 Nodes and 8 Edges

Definition 2.0.4 (Undirected Graph). : A graph is called an undirected graph if the edges do not have direction. For an edge (u, v) , the relation $(u, v) = (v, u)$ holds.

Example 2.0.5. Friendship networks are typically modeled as undirected graphs.

Definition 2.0.6 (Directed Graph). A directed graph (or digraph) is a graph whose edges have direction. An edge from vertex u to vertex v is represented as $u \rightarrow v$.

Example 2.0.7. Twitter follower networks are directed graphs because following is not always mutual.

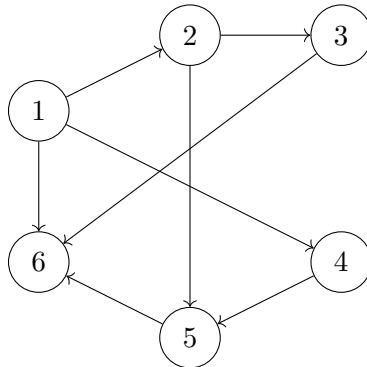


Figure 2.2: Directed Graph with 6 Nodes and 8 Edges

Definition 2.0.8 (Degree). : The degree of a vertex is the number of edges incident to the vertex. It is denoted by $deg(v)$.

Example 2.0.9. If vertex v is connected to three other vertices, then $deg(v) = 3$

Definition 2.0.10 (Adjacency Matrix). :

Let $G = (V, E)$ be a graph with n vertices. The adjacency matrix of G is defined as

$$A_{ij} = \begin{cases} 1 & \exists \text{ edge between node } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

The adjacency matrix is an $n \times n$ matrix where n is the number of vertices in the graph.

Example 2.0.11. For the graph shown in Figure 2.1, the adjacency matrix representation is given by

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

Definition 2.0.12 (Weighted Graph). : A weighted graph is a graph whose each edge has an associated numerical value called a weight. These weights may represent distance, cost, similarity, or strength of connection.

Example 2.0.13.

Definition 2.0.14 (Weighted Adjacency Matrix). : For a weighted graph, the adjacency matrix

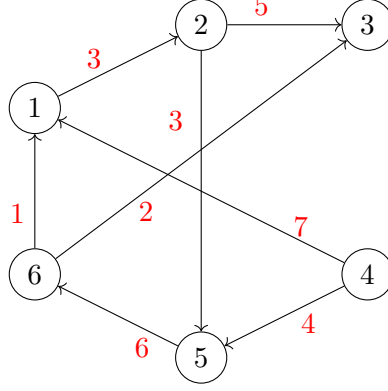


Figure 2.3: Example of a Directed Weighted Graph

contains the weights of the edges instead of binary values.

$$A_{ij} = \begin{cases} w_{ij} & \exists \text{ edge between vertices } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

where w_{ij} represents the weight of the edge between vertices i and j .

Example 2.0.15. The weighted adjacency matrix corresponding to the directed graph shown in Figure 2.3 is given by

$$A = \begin{bmatrix} 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 5 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 6 \\ 1 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

where weight of the directed edge from vertex i to vertex j is represented by w_{ij} . If no edge exists, the entry is zero.

Definition 2.0.16 (Degree Matrix). : The degree matrix of a graph is a diagonal matrix that contains the degree of each vertex on its diagonal.

$$D = \begin{bmatrix} \deg(v_1) & 0 & \dots & 0 \\ 0 & \deg(v_2) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \deg(v_n) \end{bmatrix}$$

Example 2.0.17. The degree matrix corresponding to the graph shown in Figure 2.1 is given by

$$D = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 \end{bmatrix}$$

where each diagonal element d_i represents the degree of vertex i .

Definition 2.0.18 (Graph Laplacian). : The Laplacian matrix of a graph is defined as

$$L = D - A$$

where

- D is the degree matrix
- A is the adjacency matrix

The Laplacian matrix plays an important role in spectral graph theory and graph partitioning. The Laplacian captures structural properties of the graph.

Definition 2.0.19 (Normalized Laplacian). : The normalized Laplacian matrix of a graph is defined as

$$L_{\text{norm}} = D^{-1/2} L D^{-1/2}$$

where L is the Laplacian matrix and D is the degree matrix.

Here, $D^{-1/2}$ denotes the inverse square root of the degree matrix. Since D is a diagonal matrix with node degrees on the diagonal, i.e.,

$$D = \text{diag}(d_1, d_2, \dots, d_n),$$

its inverse square root is computed element-wise as

$$D^{-1/2} = \text{diag}\left(\frac{1}{\sqrt{d_1}}, \frac{1}{\sqrt{d_2}}, \dots, \frac{1}{\sqrt{d_n}}\right).$$

Thus, $D^{-1/2}$ scales each node by the inverse square root of its degree, ensuring that contributions from neighboring nodes are normalized.

Without normalization, high-degree nodes dominate, whereas with normalization, each neighbor contributes proportionally.

Definition 2.0.20 (Connected Component). : A connected component of a graph is a collection of vertices such that there is a path between every pair of vertices in the collection and the collection is maximal with respect to this property.

If a graph has exactly one connected component. Then the graph is called connected.

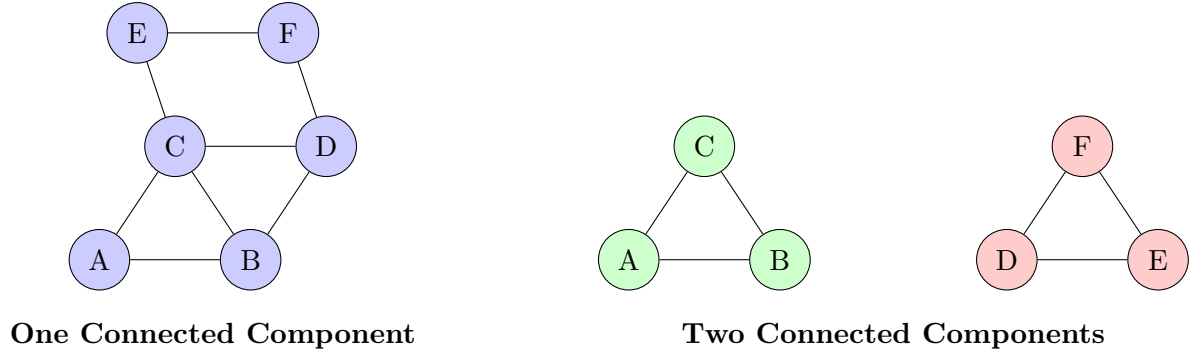


Figure 2.4: Examples of Connected Components

Example 2.0.21. Consider a graph consisting of two separate groups of vertices with no edges connecting the groups. Each group forms a connected component.

Connected Components with Same Node Labels

Graph Representation in Memory

In order to perform computational operations on a graph, such as traversal, search, and learning in graph-based models, it is necessary to store the graph efficiently in memory. The choice of representation directly affects both time and space complexity of graph algorithms.

2.1 Adjacency Matrix Representation

An adjacency matrix is a $|V| \times |V|$ matrix A , where each entry A_{ij} indicates the presence (and possibly weight) of an edge between vertices i and j .

- For an unweighted graph:

$$A_{ij} = \begin{cases} 1, & \text{if } (i, j) \in E \\ 0, & \text{otherwise} \end{cases}$$

- For a weighted graph:

$$A_{ij} = w_{ij}, \quad \text{if an edge exists, otherwise } 0$$

Space Complexity: $\mathcal{O}(|V|^2)$

Time Complexity:

- Edge lookup: $\mathcal{O}(1)$

- Iterating over neighbors: $\mathcal{O}(|V|)$

2.2 Adjacency List Representation

An adjacency list represents the graph as a collection of lists, where each vertex $u \in V$ maintains a list of its neighboring vertices. Formally, for each vertex u , we store a list:

$$\text{Adj}(u) = \{v \mid (u, v) \in E\}$$

The adjacency list corresponding to the graph shown in Figure 2.1 is given by

$$\begin{aligned} 0 &: \{1, 3, 5\} \\ 1 &: \{0, 2, 4\} \\ 2 &: \{1, 5\} \\ 3 &: \{0, 4\} \\ 4 &: \{1, 3, 5\} \\ 5 &: \{0, 2, 4\} \end{aligned}$$

Space Complexity: $\mathcal{O}(|V| + |E|)$

Time Complexity:

- Edge lookup: $\mathcal{O}(\deg(u))$
- Iterating over neighbors: $\mathcal{O}(\deg(u))$

2.3 Coordinate List (COO) Representation

For sparse graphs, storing all possible vertex pairs is inefficient. Instead, we use the Coordinate List (COO) format, which stores only the edges that actually exist.

In COO representation, the graph is stored using three arrays:

- Source nodes
- Destination nodes
- (Optional) Edge weights

Each edge $(u, v) \in E$ is stored explicitly as a tuple.

The COO representation corresponding to the graph shown in Figure 2.1 is given by

$(0, 1), (0, 3), (0, 5), (1, 0), (1, 2), (1, 4), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 3), (4, 5), (5, 0), (5, 2), (5, 4)$

Space Complexity: $\mathcal{O}(|E|)$

Thus, COO representation is highly efficient for sparse graphs, as it avoids storing non-existent edges.

In practice, many graph processing libraries internally convert graph data into more efficient formats. For instance, the **Deep Graph Library (DGL)** converts graphs into the **Compressed Sparse Row (CSR)** format during loading. CSR enables efficient neighborhood queries and fast matrix operations, which are crucial for scalable graph learning algorithms.

2.4 Compressed Sparse Row (CSR)

Compressed Sparse Row (CSR) is a memory-efficient representation of sparse matrices in which only non-zero elements are stored. It is widely used in graph processing and GPU-based computations.

CSR represents a matrix using three arrays:

- **values**: stores all non-zero elements of the matrix.
- **col_idx**: stores the column indices corresponding to each value.
- **row_ptr**: stores the starting index of each row in the values array.

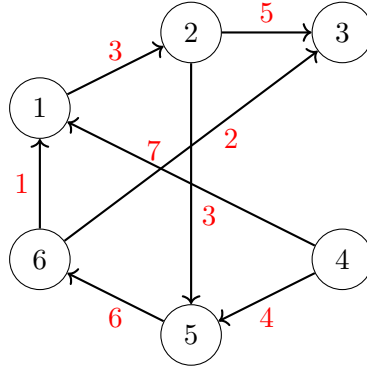


Figure 2.5: Directed weighted graph corresponding to the CSR representation.

Example 2.4.1 (CSR Representation of an Adjacency Matrix). : The adjacency matrix of the above graph figure 2.5

$$A = \begin{bmatrix} 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 5 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 6 \\ 1 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

The CSR representation is given by:

$$values = [3, 5, 3, 7, 4, 6, 1, 2]$$

$$col_idx = [2, 3, 5, 1, 5, 6, 1, 3]$$

$$row_ptr = [1, 2, 4, 4, 6, 7, 9]$$

Explanation:

- Row 1 has one non-zero element starting at index 1.
- Row 2 has two non-zero elements starting at index 2.
- Row 3 has no non-zero elements.
- Row 4 starts at index 4.
- Row 5 starts at index 6.
- Row 6 starts at index 7.

Remark 1 (Advantages of CSR). :

- **Memory Efficient:** Reduces storage from $O(N^2)$ to $O(E)$.
- **Cache Friendly:** Uses contiguous memory layout.
- **GPU Efficient:** Enables parallel processing of rows.
- **Better than Adjacency List:** Avoids pointer overhead and improves locality.

Spectral Graph Theory

Spectral graph theory studies the relationship between the structure of a graph and the eigenvalues and eigenvectors of matrices associated with the graph such as the adjacency matrix and the Laplacian matrix. These spectral properties provide important insights into connectivity, clustering, and centrality of nodes in a graph.

Definition 2.4.2 (Eigenvalue). : Let A be a square matrix. A scalar λ is called an eigenvalue of A if there exists a non-zero vector x such that

$$Ax = \lambda x$$

Definition 2.4.3 (Eigenvector). : A non-zero vector x that satisfies

$$Ax = \lambda x$$

is called an eigenvector corresponding to the eigenvalue λ .

Definition 2.4.4 (Spectrum of a Graph). : The set of eigenvalues of a matrix associated with a graph (such as the adjacency matrix or Laplacian matrix) is called the spectrum of the graph.

Theorem 2.4.5. *Let L be the Laplacian matrix of a graph with n vertices. Then*

- *All eigenvalues of L are non-negative.*
- *The smallest eigenvalue of L is 0.*

Theorem 2.4.6. *The number of zero eigenvalues of the Laplacian matrix of a graph is equal to the number of connected components of the graph.*

The eigenvectors of the adjacency matrix can be used to measure the importance of nodes in a network. This leads to the concept of eigenvector centrality, where nodes connected to highly important nodes receive higher importance scores.

2.5 Centrality Measures

Centrality measures are used to quantify the importance or influence of vertices in a network. Different centrality measures capture different aspects of node importance, such as connectivity, distance, and control over information flow within the network.

Definition 2.5.1 (Degree Centrality). The degree centrality of a vertex is defined as the number of edges incident to that vertex.

For a vertex v , the degree centrality is given by $C_D(v) = \deg(v)$, where $\deg(v)$ represents the degree of vertex v .

Example 2.5.2. In a social network, a person with many friends has a high degree centrality.

Definition 2.5.3 (Closeness Centrality). : Closeness centrality measures how close a vertex is to all other vertices in the graph.

For a vertex v , closeness centrality is defined as

$$C_C(v) = \frac{1}{\sum_{u \in V} d(v, u)}$$

where $d(v, u)$ represents the shortest path distance between vertices v and u .

Example 2.5.4. A node that can reach all other nodes with short paths will have high closeness centrality.

Definition 2.5.5 (Betweenness Centrality). : Betweenness centrality measures how frequently a vertex appears on shortest paths between other vertices.

It is defined as

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where

- σ_{st} is the total number of shortest paths between vertices s and t
- $\sigma_{st}(v)$ is the number of those paths that pass through vertex v

Example 2.5.6. In communication networks, nodes with high betweenness centrality often act as bridges connecting different parts of the network.

Definition 2.5.7 (Eigenvector Centrality). : Eigenvector centrality measures the importance of a vertex based on the importance of its neighbors.

Let A be the adjacency matrix of the graph. The eigenvector centrality score of vertex i is

defined as

$$x_i = \frac{1}{\lambda} \sum_{j=1}^n A_{ij} x_j$$

which can be written in matrix form as

$$Ax = \lambda x$$

where x is the vector of centrality scores and λ is the largest eigenvalue of the adjacency matrix.

Computational Efficiency: Power Iteration Method

Computing eigenvectors via direct eigendecomposition requires $O(n^3)$ time, which is infeasible for large-scale graphs such as OGBN-Products ($n \approx 2.4 \times 10^6$). To address this, we use the Power Iteration method to approximate the dominant eigenvector.

Let $A \in \mathbb{R}^{n \times n}$ be the adjacency matrix of the graph. The goal is to compute the eigenvector corresponding to the largest eigenvalue (in absolute value), i.e.,

$$Av = \lambda v,$$

where $\lambda_{\max}$ is the eigenvalue with the maximum magnitude.

Starting with an initial random vector $b_0 \in \mathbb{R}^n$ such that $\|b_0\| = 1$, we iteratively compute $b_{k+1} = Ab_k$, followed by normalization,

$$b_{k+1} = \frac{b_{k+1}}{\|b_{k+1}\|}.$$

As $k \rightarrow \infty$, the sequence $\{b_k\}$ converges to the eigenvector corresponding to the dominant eigenvalue $\lambda_{\max}$. Since the adjacency matrix A is sparse, each matrix-vector multiplication Ab_k requires $O(|E|)$ time, making the method scalable to large graphs.

The resulting vector represents the eigenvector centrality, where each node's importance is proportional to the importance of its neighbors.

Message Passing in Graph Neural Networks

Graph Neural Networks (GNNs) extend traditional neural networks to graph-structured data. The main mechanism used in GNNs is the message passing framework, where nodes exchange information with their neighbors to update their representations. In this framework, each node iteratively aggregates information from its neighboring nodes and updates its feature representation.

Definition 2.5.8 (Node Representation). : In Graph Neural Networks, each vertex v is associated with a feature vector h_v that represents the state or embedding of the node $h_v \in \mathbb{R}^d$ where d represents the dimension of the node feature vector.

2.6 Message Passing

Message passing is a process in which each node collects information from its neighboring nodes and updates its representation. The general message passing update rule is

$$h_v^{(k+1)} = \sigma \left(\sum_{u \in N(v)} W^{(k)} h_u^{(k)} \right)$$

where

- $h_v^{(k)}$ is the representation of node v at layer k
- $N(v)$ denotes the neighbors of node v
- $W^{(k)}$ is the learnable weight matrix
- σ is an activation function

Definition 2.6.1 (Aggregation Function). : Aggregation functions combine information from neighboring nodes during message passing.

Common aggregation functions include

- Sum aggregation
- Mean aggregation
- Max aggregation

2.7 Illustrative Example of Message Passing in GCN

To understand how Graph Convolutional Networks (GCNs) update node representations, we consider a small graph example as shown in Figure 1. The graph consists of 9 nodes, where each node is associated with a 4-dimensional feature vector $\mathbf{h} = [f_1, f_2, f_3, f_4]$. Our goal is to demonstrate how the feature vector of a specific node is updated using information from its neighboring nodes through the message passing mechanism.

We focus on node $N4$ (highlighted in the figure), whose feature update we aim to trace. Node $N4$ is connected to four neighboring nodes: $N1$, $N3$, $N5$, and $N7$. Each of these neighbors contains its own feature vector, which will contribute to updating $N4$'s representation.

The update process follows three key steps:

- **Message Passing:** Each neighboring node sends its feature vector as a message to $N4$.
- **Aggregation:** The received messages are combined using an aggregation function such as sum, mean, or max.
- **Update:** The aggregated result is used to update the feature vector of $N4$ (optionally followed by a learnable transformation and activation in GCN).

This example helps illustrate how local neighborhood information is integrated to refine node representations in graph neural networks.

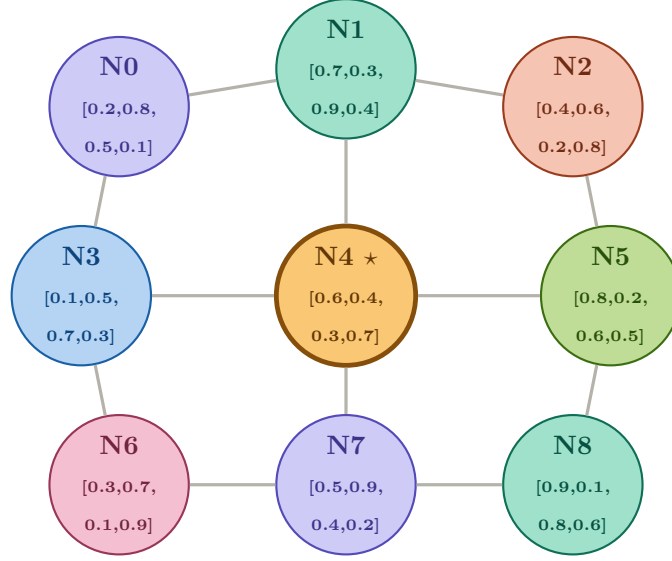


Diagram 1 — Graph structure. 9 nodes, each with a 4-dimensional feature vector $\mathbf{h} = [f_1, f_2, f_3, f_4]$. $\star$ N4 is the focus node whose update we trace.

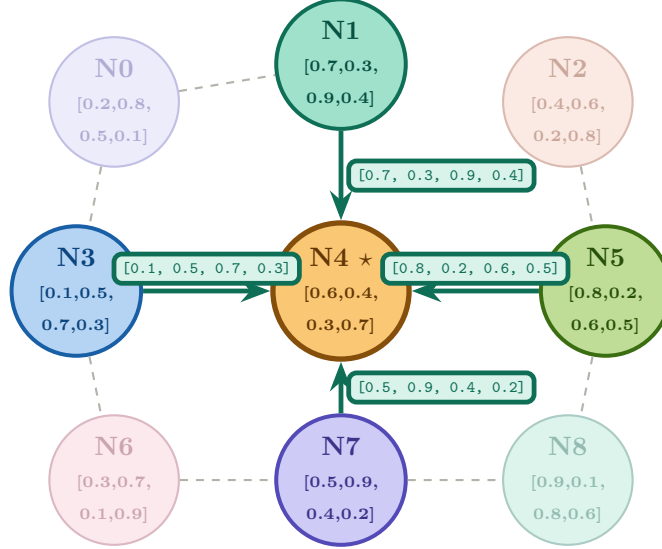


Diagram 2 — Message passing. N4's four neighbors (N1, N3, N5, N7) each send their full 4D feature vector as a “message” along the edges. Non-participating nodes are dimmed.

Aggregation (mean) — dimension by dimension:

Node	f_1	f_2	f_3	f_4
N1 (neighbor)	0.7	0.3	0.9	0.4
N3 (neighbor)	0.1	0.5	0.7	0.3
N5 (neighbor)	0.8	0.2	0.6	0.5
N7 (neighbor)	0.5	0.9	0.4	0.2
Mean ($\div 4$)	0.525	0.475	0.650	0.350

$$\mathbf{m}_4 = \frac{1}{|\mathcal{N}(4)|} \sum_{j \in \mathcal{N}(4)} \mathbf{h}_j = [0.525, 0.475, 0.650, 0.350]$$

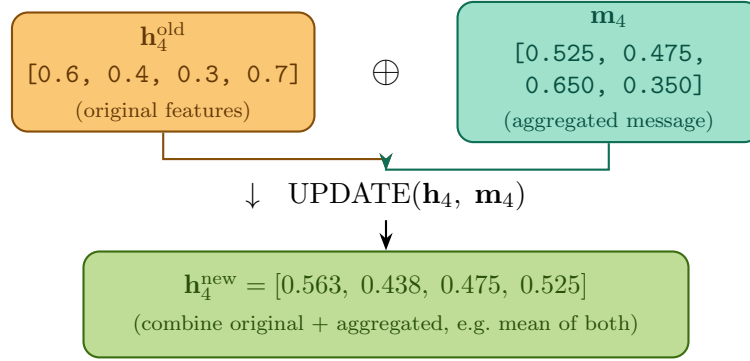


Diagram 3 — Aggregation & Update. The four neighbor vectors are averaged dimension-by-dimension to give the aggregated message $\mathbf{m}_4$. This is combined with N4's own features via the UPDATE function to produce the new feature vector $\mathbf{h}_4^{\text{new}}$. One GNN layer = **Message** $\rightarrow$ **Aggregate (mean)** $\rightarrow$ **Update**, applied simultaneously at every node.

Definition 2.7.1 (Graph Convolutional Network). : Graph Convolutional Networks apply convolution operations on graphs by combining node features with information from neighboring nodes.

The layer-wise propagation rule is $H^{(l+1)} = \sigma(\hat{A}H^{(l)}W^{(l)})$ where

- $H^{(l)}$ represents node embeddings at layer l
- $\hat{A}$ is the normalized adjacency matrix
- $W^{(l)}$ is the learnable weight matrix

Definition 2.7.2 (Centrality-Preserving Message Passing). : Centrality-preserving message passing refers to methods that ensure important nodes, as identified by centrality measures such as eigenvector centrality, maintain their influence during information propagation in the graph.

These techniques are particularly useful in graph coarsening algorithms where the structural importance of nodes must be preserved after graph reduction.

Advanced Graph Neural Network Architectures Recent developments in graph neural networks have introduced several architectures that improve the message passing framework.

Among the most widely used models are GraphSAGE and Graph Attention Networks (GAT). These models improve scalability and learning efficiency by introducing new aggregation and attention mechanisms.

2.8 GraphSAGE

GraphSAGE (Graph Sample and Aggregate) [5] is a graph neural network framework that generates node embeddings by sampling and aggregating features from a node's local neighborhood. Unlike traditional GNNs, GraphSAGE learns a function that generates embeddings by aggregating neighbor features rather than learning individual node embeddings. The update rule is given by

$$h_v^{(k+1)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k)}, \text{AGG} \left(\{h_u^{(k)}, u \in N(v)\} \right) \right) \right)$$

where

- $h_v^{(k)}$ is the representation of node v at layer k
- $\text{AGG}(\cdot)$ represents the aggregation function
- $W^{(k)}$ is the learnable weight matrix
- σ is a non-linear activation function

2.9 Graph Attention Network

Graph Attention Networks introduce an attention mechanism that assigns different importance weights to neighboring nodes during message passing. The attention coefficient between node i and node j is computed as

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_j]))}{\sum_{k \in N(i)} \exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_k]))}$$

where

- W is a learnable weight matrix
- a is the attention vector
- $\|$ denotes concatenation

The updated node representation is computed as

$$h'_i = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} Wh_j \right)$$

where α_{ij} represents the attention weight between nodes i and j .

Chapter 3

Approach

Edge Sparsification using Eigenvector Centrality

Graph Neural Networks (GNNs) rely on message passing over graph edges, where each node aggregates information from its neighbors. However, in large-scale graphs, the number of edges can be extremely high, leading to increased computational cost. Graph sparsification aims to reduce the number of edges while preserving the essential structural and spectral properties of the graph. In this work, we use **eigenvector centrality** to guide edge sparsification by identifying important nodes and preserving edges that contribute most to information flow. Nodes with high centrality are connected to other important nodes and play a key role in propagating information.

Let the original graph be defined as:

$$G = (V, E)$$

We aim to construct a sparsified graph:

$$G' = (V, E')$$

such that:

- The number of edges is reduced: $|E'| < |E|$

Thus, G' is a sparser version of G , preserving nodes but reducing connectivity.

3.1 Scalability of Eigenvector Centrality for Graph Sparsification

Training Graph Neural Networks (GNNs) on large-scale graphs is computationally expensive due to the high number of edges involved in message passing. Since each layer aggregates information from neighboring nodes, the cost scales with the number of edges, making it a major bottleneck for large graphs.

Graph sparsification addresses this challenge by reducing the number of edges while preserving the essential structural and information flow properties of the graph. This leads to:

- Reduced computational cost during message passing.

- Improved scalability to large graphs.
- Reduction of less informative connections.

3.2 Node Importance via Eigenvector Centrality

We first compute the eigenvector centrality score for each node: $A\mathbf{x} = \lambda\mathbf{x}$, where $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ and x_i represents the importance of node i .

Method 1: Eigen Decomposition

One direct method to compute eigenvector centrality is:

1. Compute all eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ of the adjacency matrix A .
2. Identify the dominant eigenvalue: $\lambda_{\max} = \max |\lambda_i|$.
3. Compute the corresponding eigenvector $\mathbf{x}$ such that:

$$A\mathbf{x} = \lambda_{\max}\mathbf{x}$$

4. Normalize the eigenvector to obtain stable centrality scores:

$$\mathbf{x} \leftarrow \frac{\mathbf{x}}{\|\mathbf{x}\|}$$

Note: The eigenvector centrality is defined by the principal eigenvector of A . In practice, this corresponds to the steady-state solution of the iterative process:

$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)}$$

with normalization at each step, converging to the dominant eigenvector.

Limitation: This approach is computationally expensive with time complexity $\mathcal{O}(n^3)$ for dense matrices, making it impractical for large-scale graphs.

Method 2: Power Iteration (Practical Approach)

Due to scalability issues, eigenvector centrality is typically computed using the power iteration method.

Algorithm

1. Initialize a non zero random vector $\mathbf{x}^{(0)} \in \mathbb{R}^n$.
2. Iteratively update:

$$\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)}$$

3. Normalize the vector at each step to prevent divergence:

$$\mathbf{x}^{(k+1)} = \frac{\mathbf{x}^{(k+1)}}{\|\mathbf{x}^{(k+1)}\|}$$

4. Repeat until convergence:

$$\mathbf{x}^{(k+1)} \approx \mathbf{x}^{(k)}$$

Starting from a random vector, repeated multiplication with the adjacency matrix amplifies the component corresponding to the largest eigenvalue. Eventually, the vector converges to the principal eigenvector.

Algorithm 1 Power Iteration for Eigenvector Centrality

- 1: Initialize b_0 as a random unit vector.
 - 2: **repeat**
 - 3: $b_{k+1} = \frac{Ab_k}{\|Ab_k\|}$
 - 4: **until** $\|b_{k+1} - b_k\| < \epsilon$
 - 5: **return** b_{k+1} as the centrality vector.
-

Since A is sparse, each iteration is $O(|E|)$, allowing us to compute importance scores for billions of edges in seconds.

Thus, the converged vector can be written as:

$$\mathbf{x} = [x_1, x_2, \dots, x_i, \dots, x_n]^T$$

where $\mathbf{x}$ is a column vector. The importance (centrality) score of node i is given by x_i . Higher values of x_i indicate that node i is more important in the graph.

Edge Scoring Mechanism

To determine the importance of an edge (i, j) , we define an edge score based on node centralities:

$$e_{ij} = x_i \cdot x_j$$

Interpretation:

- If both x_i and x_j are high $\Rightarrow$ edge connects important nodes $\Rightarrow$ high score
- If either node has low importance $\Rightarrow$ low edge score

Edge Selection Strategy

We introduce a threshold τ to decide whether to retain or remove an edge:

- If $e_{ij} \geq \tau \Rightarrow$ retain edge (i, j) in G'
- If $e_{ij} < \tau \Rightarrow$ candidate for removal

Connectivity Constraint

Even if an edge has low importance, its removal must not disrupt the connectivity of the graph. Let $\kappa(G)$ denote the number of connected components in graph G . For an edge

(i, j) to be removed, the following condition must hold:

$$\kappa(G \setminus (i, j)) = \kappa(G)$$

where $G \setminus (i, j)$ represents the graph obtained after removing the edge (i, j) .

This condition ensures that the overall connectivity of the graph remains unchanged after edge removal.

Combined Edge Removal Criterion An edge (i, j) is removed if and only if both conditions are satisfied:

$$(i, j) \in E \setminus E' \iff (e_{ij} < \tau) \text{ and } \kappa(G \setminus (i, j)) = \kappa(G)$$

where:

- E is the set of edges in the original graph,
- E' is the set of edges retained after sparsification,
- $E \setminus E'$ represents the set of removed edges.

Interpretation The above condition implies that an edge is removed only if:

1. It has low importance (i.e., $e_{ij} < \tau$), and
2. Its removal does not increase the number of connected components in the graph.

Bridge Edges and Structural Integrity If removing an edge increases the number of connected components, i.e.,

$$\kappa(G \setminus (i, j)) > \kappa(G),$$

then the edge (i, j) is called a **bridge edge** (or critical edge). Such edges are essential for maintaining graph connectivity and must be preserved regardless of their importance score.

Key Insight This strategy ensures that only non-critical and low-importance edges are removed. As a result, the graph becomes sparser while retaining its essential structural properties, which is crucial for maintaining the performance of Graph Neural Networks.

3.3 Algorithm Outline

The proposed algorithm performs graph sparsification by removing less important edges while preserving overall connectivity. Eigenvector centrality scores are first computed to capture node importance in the network.

Each edge (i, j) is assigned an importance value $e_{ij} = x_i x_j$, where edges between highly central nodes receive higher scores. Edges are then processed in ascending order of

importance, and an edge is removed if its score is below a threshold τ and its removal does not disconnect the graph.

The resulting sparsified graph $G' = (V, E')$ retains important structural information while reducing computational cost for GNN-based tasks.

Algorithm 2 Centrality-Based Edge Pruning Algorithm

```

1: Compute eigenvector centrality vector  $x$  using Power Iteration
2: for each edge  $(i, j) \in E$  do
3:    $e_{ij} = x_i \cdot x_j$ 
4: end for
5: Sort edges in ascending order of  $e_{ij}$ 
6: for each edge  $(i, j)$  in sorted order do
7:   if  $e_{ij} < \tau$  and removing  $(i, j)$  does not disconnect the graph then
8:     Remove edge  $(i, j)$ 
9:   end if
10: end for
11: return sparsified graph  $G' = (V, E')$ 

```

3.4 Advantages of the Approach

- Preserves all nodes (no information loss at node level)
- Reduces edge density significantly
- Maintains global connectivity
- Focuses on structurally important connections

3.5 Time Complexity Analysis: Original Graph vs Sparsified Graph

Message Passing Cost in GNNs

In Graph Neural Networks (GNNs), the dominant computational cost arises from the message passing (aggregation) step. For a graph $G = (V, E)$ with node feature dimension d , the cost of one GNN layer is: $\mathcal{O}(|E| \cdot d)$

Here, d denotes the **dimension of the node feature vector**, not the edge feature.

- Each node $i \in V$ has a feature vector $\mathbf{h}_i \in \mathbb{R}^d$
- For each edge $(i, j) \in E$, message passing involves processing d -dimensional node features

Thus, computation scales linearly with both the number of edges and the feature dimension.

Cost in Sparsified Graph

Now consider the sparsified graph:

$$G' = (V, E'), \quad \text{where } |E'| < |E|$$

The message passing cost becomes:

$$\mathcal{O}(|E'| \cdot d)$$

Comparison and Justification

Since:

$$|E'| < |E|$$

it directly follows that:

$$\mathcal{O}(|E'| \cdot d) < \mathcal{O}(|E| \cdot d)$$

Hence, the computational cost of running a GNN on G' is strictly lower than on G .

Memory Efficiency

Memory usage in GNNs is also proportional to the number of edges:

$$\text{Memory} \propto |E|$$

Thus:

$$\text{Memory}(G') < \text{Memory}(G)$$

This is especially important for large-scale graphs.

Preservation of Graph Structure

Despite reducing the number of edges, the sparsification process ensures:

- **Connectivity Preservation:** No increase in the number of connected components
- **Retention of Important Edges:** High-score edges ($e_{ij} = x_i x_j$) are preserved
- **Approximate Information Flow Preservation:** Key structural relationships are maintained

Overall Insight

- Original cost: $\mathcal{O}(|E| \cdot d)$
- Reduced cost: $\mathcal{O}(|E'| \cdot d)$
- Since $|E'| \ll |E|$, computation is significantly faster

- Memory usage is also reduced
- Structural integrity is preserved through careful edge selection

Conclusion: Training a GNN on the sparsified graph G' is computationally more efficient than on G , while still preserving the essential structural and informational properties of the original graph.

3.6 Message Passing Error after Edge Sparsification

Edge Importance and Pruning Rule

We define the importance of an edge (i, j) using eigenvector centrality:

$$\text{Importance}(i, j) = x_i \cdot x_j$$

Pruning Strategy: $x_i \cdot x_j < \epsilon \Rightarrow$ remove edge (i, j)

Assume that $G = (V, E)$ is the primitive graph and $G' = (V, E')$ is the sparsified graph.

Standard GNN Layer: The forward update equation for a GNN layer is defined as:

$$H^{(l+1)} = \sigma(AH^{(l)}W^{(l)})$$

For a node v , this becomes:

$$h^{(l+1)}(v) = \sigma\left(\sum_u A(v, u) h^{(l)}(u) W^{(l)}\right)$$

GNN Layer after Sparsification: After pruning, we replace A with A' :

$$h'^{(l+1)}(v) = \sigma\left(\sum_u A'(v, u) h^{(l)}(u) W^{(l)}\right)$$

Error Definition: We define the error introduced due to sparsification:

$$\Delta(v) = h^{(l+1)}(v) - h'^{(l+1)}(v)$$

Define pre-activation terms:

$$z(v) = \sum_u A(v, u) h^{(l)}(u) W^{(l)}$$

therefore,

$$z'(v) = \sum_u A'(v, u) h^{(l)}(u) W^{(l)}$$

Thus, $\Delta(v) = \sigma(z(v)) - \sigma(z'(v))$

Lipschitz Continuity of Activation: Assume the activation function σ is L -Lipschitz (true for ReLU, sigmoid):

$$\|\sigma(a) - \sigma(b)\| \leq L\|a - b\|$$

Applying this: $\|\Delta(v)\| \leq L\|z(v) - z'(v)\|$

Difference in Message Passing:

$$z(v) - z'(v) = \sum_u (A(v, u) - A'(v, u)) h^{(l)}(u) W^{(l)}$$

Let $R(v)$ denote the set of removed neighbors of v . Then:

$$z(v) - z'(v) = \sum_{u \in R(v)} A(v, u) h^{(l)}(u) W^{(l)}$$

Norm Bound: Taking norm on both sides:

$$\|z(v) - z'(v)\| \leq \sum_{u \in R(v)} A(v, u) \|h^{(l)}(u) W^{(l)}\|$$

Using matrix norm inequality:

$$\|h^{(l)}(u) W^{(l)}\| \leq \|h^{(l)}(u)\| \cdot \|W^{(l)}\|$$

Thus:

$$\|z(v) - z'(v)\| \leq \|W^{(l)}\| \sum_{u \in R(v)} A(v, u) \|h^{(l)}(u)\|$$

Final Error Bound: Substituting back:

$$\|\Delta(v)\| \leq L \cdot \|W^{(l)}\| \sum_{u \in R(v)} A(v, u) \|h^{(l)}(u)\|$$

Assume node embeddings are proportional to eigenvector centrality: $\|h^{(l)}(u)\| \propto x_u$.

Let $C = L \cdot \|W^{(l)}\|$, then

$$\|\Delta(v)\| \leq C \sum_{u \in R(v)} A(v, u) x_u$$

Using Edge Pruning Condition: Edges are removed when:

$$x_v \cdot x_u < \epsilon \quad \Rightarrow \quad x_u < \frac{\epsilon}{x_v}$$

Substituting:

$$\begin{aligned}\|\Delta(v)\| &\leq C \sum_{u \in R(v)} A(v, u) \frac{\epsilon}{x_v} \\ &= \frac{C\epsilon}{x_v} \sum_{u \in R(v)} A(v, u)\end{aligned}$$

Define $\deg_R(v)$ as the number of removed edges:

$$\|\Delta(v)\| \leq \frac{C\epsilon}{x_v} \cdot \deg_R(v)$$

Global Error Bound: In matrix form:

$$\|H^{(l+1)} - H'^{(l+1)}\| \leq L \cdot \|W^{(l)}\| \cdot \|A - A'\| \cdot \|H^{(l)}\|$$

Chapter 4

Datasets and Experimental Setup

We evaluate our proposed edge sparsification method on four widely used benchmark graph datasets:

- **Reddit:** A large-scale social network dataset where nodes represent posts and edges represent interactions. It is characterized by high connectivity and large average degree.
- **OGBN-Products:** A product co-purchasing network from the Open Graph Benchmark (OGB), where nodes represent products and edges indicate that two products are frequently bought together.
- **OGBN-Arxiv:** A citation network where nodes correspond to research papers and edges denote citation relationships. This graph is relatively sparse compared to Reddit.
- **Yelp:** A business review network where nodes represent businesses and edges indicate relationships such as shared reviewers or interactions.

4.1 Dataset Statistics

The datasets vary significantly in terms of size and connectivity. A key structural property is the **average degree**, defined as:

$$\text{Average Degree} = \frac{2|E|}{|V|} \quad (4.1)$$

Dataset	No. of Vertices	No. of Edges	Average Degree
Reddit	232,965	57,424,428	493
OGBN-Products	2,449,029	63,083,526	52
OGBN-Arxiv	169,343	1,242,470	15
Yelp	45,954	3,869,956	168

Table 4.1: Graph statistics of datasets

Observation:

- Reddit is extremely dense, making it ideal for sparsification.
- OGBN-Arxiv is sparse, so aggressive pruning may harm performance.
- Yelp and OGBN-Products lie in the moderate regime.

4.2 Experimental Setup

All experiments were conducted on a remote high-performance computing (HPC) cluster accessed via SSH. The cluster follows a master–compute node architecture, where the master node serves as the login and development node, and dedicated compute nodes are used for execution. The experiments were performed on the compute node `gpu1`.

The compute node is equipped with an Intel(R) Xeon(R) Gold 5218 CPU operating at 2.30 GHz, featuring 16 physical cores and 32 threads. The system is based on the `x86_64` architecture with a single socket configuration and supports advanced vector extensions such as AVX2 and AVX-512.

The node is powered by an NVIDIA RTX A6000 GPU with 48 GB of memory. The GPU runs with NVIDIA driver version 550.54.14 and CUDA version 12.4, enabling efficient GPU-accelerated computation for large-scale experiments.

The proposed graph coarsening method was implemented in Python (version 3.11.4) within a virtual environment. For developing and training Graph Neural Network (GNN) models, we used PyTorch version 2.3.0 (CUDA 12.1) and DGL (Deep Graph Library) version 2.5.0 (CUDA 12.1). All experiments leveraged GPU acceleration to ensure scalability and computational efficiency.

- Original graph $G = (V, E)$ (Baseline)
- Sparsified graph $G' = (V, E')$

Two sparsification strategies are used:

- **Average Degree Sparsification:** Retaining top average number of edges.
- **Top-K Sparsification:** Retaining top- k important edges.
- **Threshold-based Sparsification:** Removing edges with $x_i x_j < \epsilon$.

Evaluation metrics:

- Accuracy
- Training Time
- Number of Edges

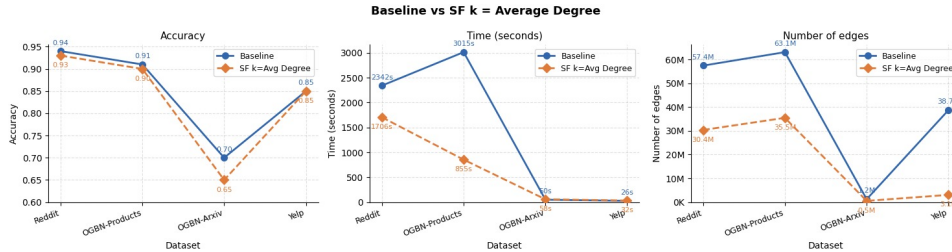
4.3 Average Degree Sparsification

We analyze sparsification by computing the average degree of the bidirectional graph and using it as an adaptive threshold for neighbourhood selection.

Key Observations

- **Degree-Adaptive Pruning:** For each node, the average degree of the full graph is computed and used as the value of k . Each node then retains only its top- k neighbours ranked by centrality score, directly tying the sparsification level to the graph’s natural connectivity.
- **Bidirectional Awareness:** The average degree is computed over the full bidirectional graph before pruning, ensuring that the symmetric structure of edge relationships is taken into account when determining how many neighbours each node retains.

Dataset	Baseline			SF $k = \text{Average Degree}$		
	Acc.	Time	Edges	Acc.	Time	Edges
Reddit	.94	2342.29s	57,424,428	.93	1706.31s	30,357,684
OGBN-Products	.91	3015.11s	63,083,526	.90	854.65s	35,463,126
OGBN-Arxiv	.70	49.69s	1,242,470	.65	57.54s	542,613
Yelp	.85	26.25s	3,869,956	.85	32.46s	3,067,716



4.4 Results: Top-K Sparsification

We first analyze sparsification by selecting top- k important edges based on centrality scores.

Key Observations

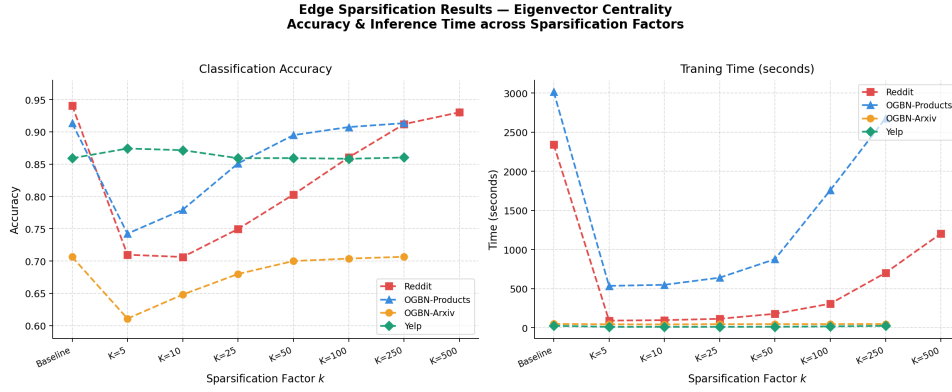
- **Significant Edge Reduction:** For all datasets, the number of edges decreases drastically as k decreases.
- **Computation Time Reduction:** Training time reduces proportionally with the number of edges, confirming the theoretical complexity $\mathcal{O}(|E|.d)$.
- **Accuracy Trade-off:**

- For dense graphs (e.g., Reddit), accuracy remains stable even after heavy sparsification.
- For sparse graphs (e.g., OGBN-Arxiv), accuracy drops more quickly.

Table 4.2: Edge sparsification results using eigenvector centrality across different sparsification factors. Each entry reports classification accuracy (%), inference time (seconds), and number of remaining edges.

Dataset	Baseline			SF $k = 5$			SF $k = 10$			SF $k = 25$			SF $k = 50$			SF $k = 100$			SF $k = 250$			SF $k = 500$		
	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges
Reddit	.94	2342.29s	57,424,428	.70	92.26s	689,653	.71	97.81s	1,244,952	.75	115.30s	2,835,497	.80	179.04s	5,291,891	.86	308.17s	9,649,972	.91	703.10s	19,691,977	.93	1204.02s	30,598,452
OGBN-Products	.91	3015.11s	63,083,526	.74	536.25s	679,472	.78	549.51s	11,450,608	.85	640.51s	22,327,577	.89	877.31s	34,705,249	.90	1758.35s	47,342,947	.91	2678.49s	57,076,419	-	-	-
OGBN-Arxiv	.70	49.69s	1,242,470	.61	44.90s	417,553	.64	43.33s	608,212	.68	46.65s	871,507	.70	46.94s	1,001,256	.70	47.39s	1,078,980	.70	48.72s	1,144,817	-	-	-
Yelp	.85	26.25s	3,869,956	.87	12.66s	137,598	.87	12.89s	251,340	.85	12.87s	585,767	.85	14.05s	1,112,996	.85	18.54s	2,026,430	.86	25.44s	3,626,997	-	-	-

SF = Sparsification Factor k ; Acc. = classification accuracy; Time = total inference time (seconds); Edges = number of edges remaining after sparsification. “-” indicates result not available or not applicable.



4.5 Results: Threshold-based Sparsification

We now analyze sparsification using threshold ϵ .

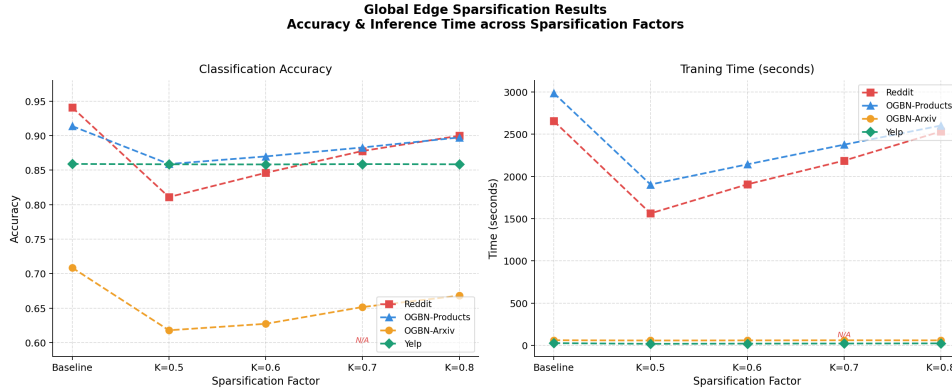
Key Observations

- **Controlled Edge Removal:** Increasing ϵ removes more edges, leading to a sparser graph.
- **Efficiency Gains:** Training time reduces significantly with increasing sparsification.
- **Performance Stability:**
 - For moderate ϵ , accuracy remains comparable to baseline.
 - Very high ϵ leads to performance degradation due to excessive pruning.

Table 4.3: Edge sparsification results using degree-based centrality across different sparsification factors. Each entry reports classification accuracy (%), inference time (seconds), and number of remaining edges.

Dataset	Baseline			Sparsification (0.5)			Sparsification ($K=0.6$)			Sparsification ($K=0.7$)			Sparsification ($K=0.8$)		
	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges	Acc.	Time	Edges
Reddit	.94	2656.42s	57,424,428	.81	1562.73s	28,770,455	.84	1906.34s	34,501,250	.87	2187.54s	40,232,044	.90	2532.62s	45,962,839
OGBN-Products	.91	2987.80s	63,083,526	.86	1904.79s	32,154,014	.87	2143.35s	38,339,912	.88	2376.72s	44,525,811	.90	2602.76s	50,711,714
OGBN-Arxiv	.70	60.47s	1,242,470	.62	57.49s	663,571	.63	58.64s	779,350	.66	60.09s	895,130	.66	59.17s	1,010,910
Yelp	.85	26.13s	3,869,956	.85	18.17s	1,946,466	.85	20.60s	2,331,164	.85	21.77s	2,715,862	.86	23.35s	3,100,560

Acc. = classification accuracy; Time = total inference time (seconds); Edges = number of edges remaining after sparsification.



4.6 Effect of Edge Sparsification on Model Accuracy

Figure 4.1 illustrates the relationship between the number of edges and classification accuracy under both **local** and **global** sparsification strategies across multiple datasets.

Key Observation

The results clearly demonstrate that the classification accuracy of Graph Neural Networks (GNNs) is strongly dependent on the number of edges retained in the graph.

$$\text{Accuracy} \propto |E'| \quad (4.2)$$

where $|E'|$ is the number of edges after sparsification.

Trend Analysis

- **Increasing Edges Improves Accuracy:** As the number of edges increases, the model has access to richer neighborhood information, leading to improved accuracy.
- **Diminishing Returns:** After a certain point, increasing the number of edges yields marginal improvements, indicating redundancy in dense graphs.
- **Effect of Sparsification:** Aggressive sparsification (very low $|E'|$) leads to a noticeable drop in accuracy due to loss of important structural information.

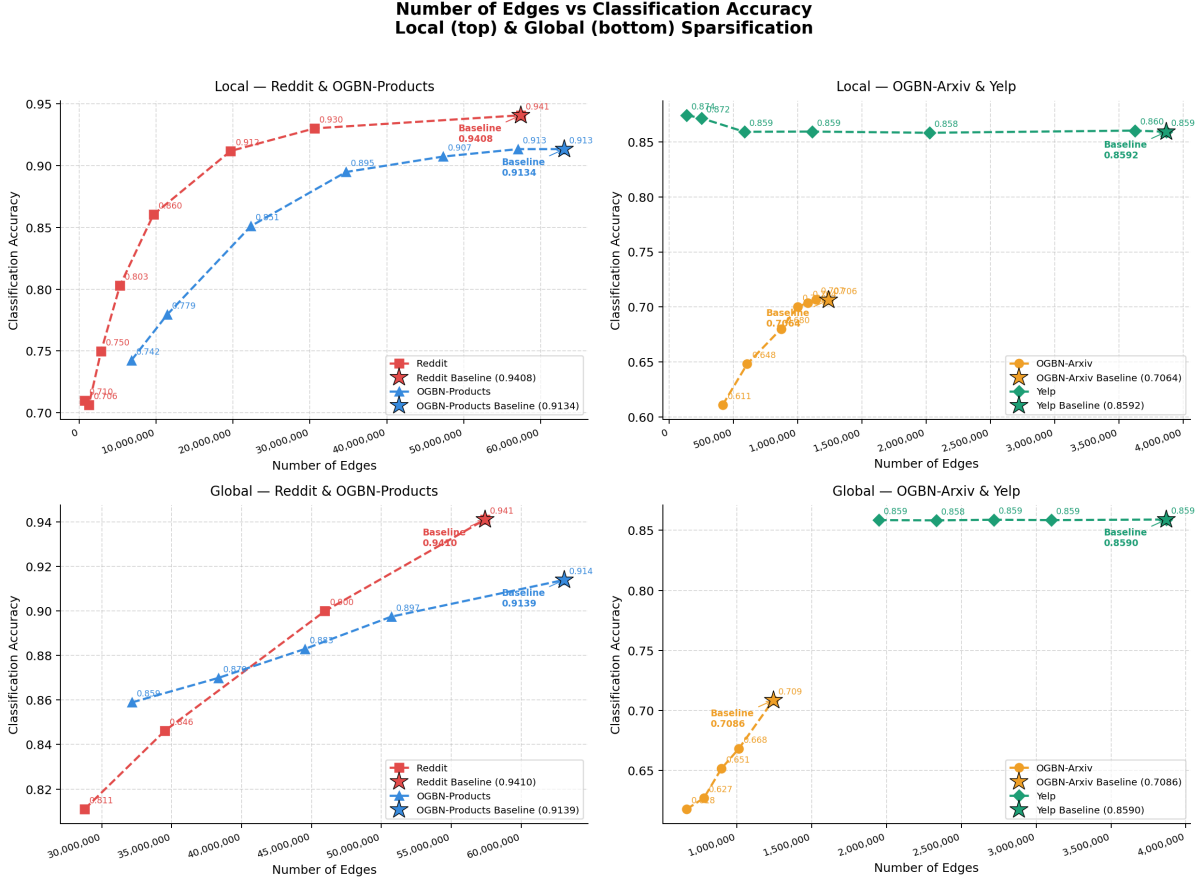


Figure 4.1: Number of Edges vs Classification Accuracy for Local (top) and Global (bottom) Sparsification

Dataset-Specific Behavior

- **Reddit & OGBN-Products (Dense Graphs):** These datasets show high resilience to edge removal. Even with significant sparsification, the accuracy remains close to baseline, indicating redundancy in edges.
- **OGBN-Arxiv (Sparse Graph):** Accuracy is more sensitive to edge removal, as fewer edges carry more critical information.
- **Yelp (Moderately Dense):** Accuracy remains relatively stable across different sparsification levels, suggesting a balance between redundancy and essential connectivity.

Local vs Global Sparsification

- **Local Sparsification:** Provides finer control over edge selection, often preserving important local structures, leading to stable accuracy.
- **Global Sparsification:** Applies uniform pruning across the graph, which may remove some locally important edges but still maintains overall performance.

Key Insight

Not all edges contribute equally to model performance; a carefully selected subset of edges can achieve near-baseline accuracy with significantly reduced computational cost.

Conclusion from Figure

The figure validates that:

- Model accuracy increases with the number of retained edges
- However, beyond a threshold, additional edges contribute little improvement
- Efficient sparsification can reduce edges drastically while maintaining high accuracy

Thus, edge sparsification enables a favorable trade-off between **efficiency** and **performance**.

4.7 Code Availability

The implementation of the proposed graph sparsification method is openly available at:

[https://github.com/himukyd/
Graph-Sparsification-for-High-Performance-Graph-Neural-Networks](https://github.com/himukyd/Graph-Sparsification-for-High-Performance-Graph-Neural-Networks)

Chapter 5

Related Work

Graph sparsification and Graph Neural Network (GNN) acceleration have attracted substantial research interest in recent years, motivated by the growing scale of real-world graph data. In this chapter, we survey the most relevant prior work across four broad themes:

- (i) Classical graph sparsification methods.
- (ii) Sampling-based approaches for scalable GNN training.
- (iii) Centrality-driven graph reduction techniques.
- (iv) Spectral and coarsening-based methods.

We situate our proposed approach within this and highlight the key distinctions.

5.1 Classical Graph Sparsification

The problem of constructing a sparse approximation of a graph while preserving certain structural properties has been studied extensively in the theoretical computer science community, well before the rise of Graph Neural Networks.

Spectral Sparsifiers. Spielman and Teng [16] introduced the notion of a spectral sparsifier: a subgraph $G' = (V, E')$ that approximates the original graph $G = (V, E)$ with respect to all cuts and quadratic forms of the Laplacian. Formally, a $(1 + \varepsilon)$ -spectral sparsifier satisfies:

$$(1 - \varepsilon) \mathbf{x}^T L_G \mathbf{x} \leq \mathbf{x}^T L_{G'} \mathbf{x} \leq (1 + \varepsilon) \mathbf{x}^T L_G \mathbf{x} \quad \forall \mathbf{x} \in \mathbb{R}^n$$

where L_G and $L_{G'}$ are the graph Laplacians of G and G' , respectively. Building on this, Spielman and Srivastava [14] showed that every graph on n vertices has a spectral sparsifier with $O(n \log n / \varepsilon^2)$ edges, constructable in nearly-linear time using random sampling weighted by effective resistances. While this line of work provides strong theoretical guarantees, these methods were designed for preserving Laplacian quadratic forms and are

not directly tailored for the message-passing objectives of GNN training.

Cut Sparsifiers. Benczúr and Karger [1] proposed cut sparsifiers that preserve all cuts of a graph up to a multiplicative $(1 + \varepsilon)$ factor with $O(n \log n)$ edges. Their sampling probability is proportional to the inverse of the edge’s *strong connectivity strength*, which is related to effective resistance. Although elegant and widely influential, this approach does not account for node feature information or the specific aggregation patterns required in GNN message passing.

Effective Resistance Sampling. Effective resistance has emerged as a principled measure of edge importance in both spectral sparsification and graph machine learning. Spielman and Srivastava [14] demonstrated that sampling edges proportional to their effective resistances yields high-quality spectral sparsifiers. Li and Schild [15] further improved the running time to near-linear. However, computing exact effective resistances requires solving linear systems, which remains costly at the scale of graphs considered in modern GNN benchmarks such as OGBN-Products (with over 63 million edges). In contrast, our approach relies on eigenvector centrality computed via the power iteration method, which requires only $O(|E|)$ per iteration and scales naturally to such graphs.

5.2 Sampling-Based Methods for Scalable GNN Training

A parallel line of work focuses on making GNN training tractable on large graphs through various forms of stochastic sampling, rather than pre-processing the graph structure.

GraphSAGE. Hamilton, Ying, and Leskovec [6] proposed GraphSAGE (Graph Sample and Aggregate), a foundational framework for inductive GNN training. Instead of computing embeddings over the full neighborhood, GraphSAGE samples a fixed-size set of neighbors at each layer. The update rule is:

$$h_v^{(k+1)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k)}, \text{AGG} \left(\{h_u^{(k)} : u \in \hat{\mathcal{N}}(v)\} \right) \right) \right)$$

where $\hat{\mathcal{N}}(v)$ is a sampled subset of the neighbors of v . While GraphSAGE effectively reduces the neighborhood size per layer, it operates on the original graph at training time and does not reduce the global graph structure. Our approach is complementary: we pre-process the graph to eliminate structurally unimportant edges, which reduces the total edge count seen during all training iterations, not just within a single forward pass.

FastGCN. Chen, Ma, and Xiao [2] proposed FastGCN, which interprets GNN layers as integral transforms of embedding functions under probability measures and samples nodes at each layer independently. This allows batch training with a fixed computation cost per layer. However, independent layer-wise sampling can lead to inconsistent neighborhoods across layers, sometimes degrading convergence. By contrast, our sparsification approach modifies the graph structure once before training and provides a consistent reduced graph for all training epochs.

Cluster-GCN. Chiang et al. [3] proposed Cluster-GCN, which partitions the graph into clusters using algorithms such as METIS, and then samples one or more clusters per mini-batch. Training proceeds on the induced subgraph of the sampled cluster, ensuring that all sampled neighbors are within the mini-batch. While effective for reducing memory footprint and improving throughput, this approach still trains on the full intra-cluster edges without considering individual edge importance. Our method provides a complementary preprocessing step that could be applied before or after clustering to further reduce the edge set.

LADIES and AS-GCN. Zou et al. [21] proposed LADIES (LAYER-Dependent Importance Sampling), which samples neighbors in a layer-dependent manner to reduce variance. Liu et al. proposed AS-GCN (Adaptive Sampling), which uses policy gradients to learn optimal sampling distributions during training. These methods are adaptive and data-driven but introduce additional model complexity and are not directly applicable to the problem of preprocessing-based graph reduction.

5.3 Centrality-Based Graph Reduction

Centrality measures have long been used in network analysis to quantify node importance, and several works have explored their utility for graph summarization and compression.

PageRank and Influence-Based Pruning. PageRank [9], developed by Brin and Page, is closely related to eigenvector centrality. Both methods define node importance recursively: a node is important if it is linked to by other important nodes. PageRank introduces a damping factor to model random restarts, while eigenvector centrality is defined as the dominant eigenvector of the adjacency matrix directly. Several works have used PageRank scores to rank and prune nodes or edges in graphs, particularly in web-scale link analysis. Our work draws on this rich tradition but adapts the eigenvector centrality formulation specifically for the GNN message-passing setting, providing a formal error bound on the impact of edge removal on node representations.

Betweenness and Bridgeness-Based Pruning. Betweenness centrality measures how often a node (or edge) lies on shortest paths between other nodes. Several graph simplification works [4] have proposed removing edges with high betweenness to detect communities, or retaining high-betweenness edges to maintain connectivity. However, computing exact betweenness centrality requires $O(|V||E|)$ time using Brandes’ algorithm, which is infeasible for graphs with millions of nodes. Our approach based on eigenvector centrality, computed via power iteration in $O(|E|)$ per iteration, offers a computationally superior alternative while still capturing the global structural importance of nodes.

Degree-Based Pruning. The simplest centrality-based pruning strategy assigns importance to edges based solely on the degrees of their endpoints. Velickovic et al. [17] and related works have observed that high-degree nodes often dominate message passing aggregation. Degree-based thresholding is computationally attractive ($O(|E|)$) but

ignores the network topology beyond immediate neighborhoods. Our eigenvector centrality approach naturally subsumes degree centrality as a special case (in regular graphs, both measures coincide) but provides a richer, globally-informed importance score.

DropEdge. Rong et al. [12] proposed DropEdge, a technique that randomly removes edges from the graph at each training epoch as a data augmentation and regularization strategy to reduce over-smoothing. While DropEdge is easy to implement, it treats all edges as equally likely to be dropped, ignoring structural importance. Our approach provides a principled, non-random alternative: edges are ranked by eigenvector centrality scores $e_{ij} = x_i \cdot x_j$ and removed deterministically, ensuring that structurally critical edges are always retained.

5.4 Summary and Positioning of Our Work

In contrast to spectral sparsifiers, which provide Laplacian approximation guarantees but are computationally expensive, our method uses eigenvector centrality computed via power iteration and runs in $O(k \cdot |E|)$ time (where k is the number of iterations, typically small). Unlike sampling-based approaches (GraphSAGE, FastGCN, ClusterGCN), our method permanently reduces the graph structure before training, yielding consistent speedup across all training epochs without per-epoch randomness. Unlike DropEdge, our pruning is deterministic and guided by global structural importance rather than uniform random chance. Unlike graph pooling methods (DiffPool, SAGPool), we do not merge or eliminate nodes, preserving the full node set and making our method directly applicable to node-level prediction tasks. Finally, compared to pure degree-based pruning, our eigenvector centrality criterion captures multi-hop structural information and has a well-defined mathematical foundation in spectral graph theory.

To the best of our knowledge, no prior work has proposed using eigenvector centrality as a principled, scalable criterion for preprocessing-based edge sparsification in GNN training, together with a formal bound on the resulting message-passing error. This positions our contribution as both novel and practically significant for large-scale graph learning.

Chapter 6

Conclusion

In this thesis, we proposed a principled and efficient approach for graph sparsification based on eigenvector centrality, aimed at improving the scalability of Graph Neural Networks (GNNs).

The core idea is to identify structurally important edges using node centrality scores and remove edges that contribute minimally to information propagation. By defining edge importance as:

$$e_{ij} = x_i \cdot x_j$$

we ensure that edges connecting highly important nodes are preserved, while less significant edges are pruned.

6.1 Methodological Contributions

The proposed framework introduces:

- A centrality-driven edge scoring mechanism
- A pruning strategy based on a threshold ϵ
- A connectivity-preserving constraint to avoid graph disconnection

Unlike naive edge removal, our approach carefully balances sparsity and structural integrity, ensuring that important graph properties are retained.

6.2 Computational Efficiency

We showed that the computational complexity of GNNs is directly proportional to the number of edges:

$$\mathcal{O}(|E| \cdot d)$$

By constructing a sparsified graph $G' = (V, E')$ with $|E'| \ll |E|$, we achieve:

- Significant reduction in training time
- Lower memory consumption
- Improved scalability to large graphs

These improvements make the approach highly suitable for real-world large-scale graph applications.

6.3 Empirical Observations

Extensive experiments on benchmark datasets (Reddit, OGBN-Products, OGBN-Arxiv, Yelp) validate our approach:

- Dense graphs benefit significantly from sparsification with minimal accuracy loss
- Moderate sparsification achieves a strong trade-off between efficiency and performance
- Sparse graphs are more sensitive to aggressive pruning

The results confirm that centrality-based sparsification effectively reduces redundancy in graphs.

Key Insight

The central insight of this work is that:

Not all edges contribute equally to message passing in GNNs.

By leveraging eigenvector centrality, we identify and retain only the most informative connections, enabling efficient learning without significant loss of information.

Limitations

Despite its effectiveness, the proposed method has certain limitations:

- Sensitivity to threshold selection (ϵ)
- Potential performance degradation in highly sparse graphs
- Preprocessing cost for computing centrality

These factors should be carefully considered in practical applications.

Future Work

Future research directions include:

- Adaptive threshold selection strategies
- Integration with learnable sparsification mechanisms
- Extension to dynamic and heterogeneous graphs
- Combining with spectral sparsification techniques

Final Remark

In conclusion, this thesis demonstrates that eigenvector centrality-based edge sparsification is a simple yet powerful approach for reducing graph complexity.

It provides a strong balance between:

- Efficiency (reduced computation)
- Effectiveness (preserved accuracy)

making it a promising direction for scalable and efficient Graph Neural Network design.

Bibliography

- [1] András A Benczúr and David R Karger. Approximating st minimum cuts in $\tilde{O}(n^2)$ time. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 47–55, 1996.
- [2] Jie Chen, Tengfei Ma, and Cao Xiao. Fastgcn: fast learning with graph convolutional networks via importance sampling. *arXiv preprint arXiv:1801.10247*, 2018.
- [3] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 257–266, 2019.
- [4] Michelle Girvan and Mark EJ Newman. Community structure in social and biological networks. *Proceedings of the national academy of sciences*, 99(12):7821–7826, 2002.
- [5] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- [6] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- [7] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [8] Ziqi Liu, Chaochao Chen, Xinxing Yang, Jun Zhou, Xiaolong Li, and Le Song. Heterogeneous graph neural networks for malicious account detection. In *Proceedings of the 27th ACM international conference on information and knowledge management*, pages 2077–2085, 2018.
- [9] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical report, Stanford infolab, 1999.
- [10] Namyong Park, Andrey Kan, Xin Luna Dong, Tong Zhao, and Christos Faloutsos. Estimating node importance in knowledge graphs using graph neural networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 596–606, 2019.

- [11] Sungmin Rhee, Seokjun Seo, and Sun Kim. Hybrid approach of relation network and localized graph convolutional filtering for breast cancer subtype classification. *arXiv preprint arXiv:1711.05859*, 2017.
- [12] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Droppedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.
- [13] Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, pages 135–146, 2013.
- [14] Daniel A Spielman and Nikhil Srivastava. Graph sparsification by effective resistances. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 563–568, 2008.
- [15] Daniel A Spielman and Shang-Hua Teng. Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems. In *Proceedings of the thirty-sixth annual ACM symposium on Theory of computing*, pages 81–90, 2004.
- [16] Daniel A Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM (JACM)*, 51(3):385–463, 2004.
- [17] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- [18] Wei Wu, Bin Li, Chuan Luo, and Wolfgang Nejdl. Hashing-accelerated graph neural networks for link prediction. In *Proceedings of the Web Conference 2021*, pages 2910–2920, 2021.
- [19] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, 32(1):4–24, 2020.
- [20] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 974–983, 2018.
- [21] Difan Zou, Ziniu Hu, Yewen Wang, Song Jiang, Yizhou Sun, and Ququan Gu. Layer-dependent importance sampling for training deep and large graph convolutional networks. *Advances in neural information processing systems*, 32, 2019.